

MCP como patrón de conectores

Claude Certified Developer, Foundation (CCDV-F) · Masterclass UST

ES

Sesión 4 de 12 · Notas en español (es-ES)

El último tercio de la sesión 4 presentó **MCP (Model Context Protocol)** como la forma en que los clientes Claude enganchan sistemas de terceros. El ejemplo en directo no fue una clase abstracta de protocolo: activar el servidor MCP de Figma, autenticar, inspeccionar la lista de tools, pasar una URL de Figma a una skill de frontend y dejar que Claude Code genere una UI React/Vite/TypeScript desde el contexto de diseño. Construir servidores MCP propios se deja para los módulos de API; aquí hace falta el patrón de conector.

PROTOCOLO FRENTE AL DEMO DE FIGMA

Los nombres del marketplace, las pantallas OAuth y la lista exacta de tools (get screenshot, get design context, get metadata...) cambiarán. Las ideas que permanecen: MCP expone *herramientas* sobre las que el modelo puede planificar; dais consentimiento por invocación; los resultados caen como ficheros temporales en un sandbox; seguís escribiendo skills y CLAUDE.md para que la generación respete el stack. Docs vigentes: [MCP](#) y [tool use](#).

Por qué existe MCP en este curso

Un modelo solo ve lo que ponéis en el prompt, adjuntáis como ficheros o recuperáis vía tools. Las skills y CLAUDE.md no pueden iniciar sesión en Figma. MCP es la vía normalizada para decir: este servidor ofrece operaciones (tools); Claude puede llamarlas; el host (Claude Code / Claude.ai / Cowork) media autenticación y sandbox. La versión en una frase del formador: *obtener detalles de terceros con un protocolo llamado MCP* — Figma hoy, datos internos más adelante cuando implementéis servidores vosotros.

- No es obligatorio para todo frontend: podríais pegar el HTML de Cowork Design en Code. El salto a Figma fue explícitamente «quiero mostrar MCP».
- Permisos especiales: hay que activar/conectar las tools MCP; un servidor desconectado no aparece en el plan.
- La misma cuenta Claude y la de Figma deben completar el OAuth/consentimiento para emitir un token al cliente.

Conectar un servidor (Figma en el laboratorio)

1. En Claude Code (o el panel MCP que mostraba la UI), abrid MCP servers e instalad/conectad el servidor Figma.
2. Autenticad: el cliente abre una URL en el navegador; aprobáis el acceso (correo, token). Usad el perfil que realmente posee el fichero de Figma.
3. Confirmad el estado: connected. Volved a «MCP servers» si la UI sigue en desconectado.

4. Abrid la lista de tools. En la grabación: get screenshot, get design context, get motion context, get metadata, get diagrams, exportar vídeo, mappings de código, y similares. No memoricéis la lista; entendid que *el modelo planifica qué tools llamar*.
5. Copiad la *URL concreta del fichero* en Figma. Un prompt genérico «usa mi Figma» puede recorrer toda la cuenta y malgastar tokens.

En Figma quizá tengáis que crear un token si el conector lo pide; el demo usó sobre todo OAuth tras entrar con el perfil de Google correcto. Compartid el fichero si la identidad de Claude no es ya colaboradora.

Skill de frontend + memoria + MCP en un solo prompt

Antes del comando de generar, el formador añadió una **skill de frontend** (ingeniero frontend senior; React, TypeScript, Vite; componentes responsivos, accesibles, reutilizables; manejo de errores; routing; Next.js opcional si ese es el stack de casa) y un **CLAUDE .md** de frontend (asignación de tareas, prioridades, «respetar este OpenAPI», principios de UI). Luego una *sesión nueva*, slash de frontend y un prompt del tipo: genera el frontend a partir del diseño de Figma en esta URL.

Los primeros movimientos de Claude son MCP, no JSX: «¿tengo servidor Figma? ¿qué tools? get metadata, get design context...» Cada llamada es una function call. Veréis prompts de consentimiento («¿continuar?»). Aprobados con la misma seriedad que un comando de shell.

Dónde viven los resultados: sandbox, no el chat para siempre

Un alumno preguntó si el volcado de metadatos de Figma era un fichero temporal. Sí. Los resultados MCP/tool se guardan como artefactos temporales: metadatos a menudo como `.txt`, APIs JSON como JSON, en un **sandbox** del espacio de trabajo local/Claude. La sesión anterior ya había presentado ese sandbox; esta lo reutiliza. Pensad en un entorno aislado (el formador lo comparó con un SO tipo Docker donde se puede instalar Node o Python — analogía, no «MCP ejecuta Docker»). El LLM inyecta listas de ficheros/carpetas; el IO del sandbox escribe el árbol (estilo code interpreter). Tras el primer chorro de MCP, la generación usa ese contexto de diseño en caché en vez de volver a pedir cada componente.

Lo que veis en la UI	Qué suele ser	Qué debéis hacer
Consentimiento a <code>get_metadata / get_design_context</code>	Llamada a tool del servidor MCP	Leed el nombre de la tool; denegad si la URL es incorrecta.
<code>txt/json</code> temporal en el sandbox	Payload del resultado	No subáis secretos; tratadlo como efímero.
Andamiaje (Vite, React, Tailwind, <code>src/</code> , <code>package.json</code>)	IO local tras el plan	Revisad <code>gitignore</code> ; no subáis <code>node_modules</code> .
Avisos de desajuste (pantalla UI vs API ausente)	El modelo compara diseño y OpenAPI	Omitid, añadid un endpoint o caed la pantalla — no lo ignoréis en silencio en producción.

Notas de tokens y uso del arranque en directo

La generación frontend de extremo a extremo en la grabación se quedó en torno al 10–12 % del medidor de uso del formador para ese tamaño de app. Los productos grandes hay que trocearlos. La verificación (abrir el navegador, comprobar la UI) puede sumar diez minutos o más: por eso la sesión se detuvo a medias y se reanudó el 12 de agosto. Preferid contexto de diseño estructurado (estilo Mermaid) frente a capturas gigantes cuando podáis; el «get design context» de MCP ya intenta dar estructura, no solo píxeles.

Lo que MCP aún no es en este capítulo

- No sustituye a CLAUDE.md ni a las skills: esas siguen acotando stack y calidad cuando llega el contexto.
- No es escritura automática en Figma; es lectura/contexto de diseño para generar, salvo que activéis otras tools.
- No es vuestro origen de datos interno UST. Los servidores MCP internos llegan cuando defináis tools en el lado API.
- No es motivo para saltarse OpenAPI: el modelo siguió detectando endpoints de gestión de usuarios ausentes frente a una pantalla de listado.

Documentación oficial

- **MCP**

<https://docs.claude.com/en/docs/agents-and-tools/mcp>

Patrón de conectores: servidores, tools y cómo Claude los aloja.

- **Visión general de tool use**

<https://docs.claude.com/en/docs/agents-and-tools/tool-use/overview>

Forma de function calling sobre la que se sientan las tools MCP.

- **Visión general de Claude Code**

<https://code.claude.com/docs/en/overview>

Dónde se activan los servidores MCP en el cliente de código.

- **Inicio de Claude Docs**

<https://docs.claude.com/en/home>

Si la UI del conector Figma se ha movido.

Study notes distilled from the UST CCD Foundation masterclass. Prefer Anthropic's current docs over session anecdotes. Not official exam material.